SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

Django

Form

Python

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

Django Form

- Form class
- Widgets

Django ModelForm

- Meta class
- ModelForm 적용
- save 메서드

HTTP 요청 다루기

- View 함수 구조 변화
- new & create 함수 결합
- edit & update 함수 결합

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

참고

- ModelForm의 키워드 인자 구성
- Widgets 응용
- 필드를 수동으로 렌더링

학습 시작

웹사이트에서 회원가입이나 글을 쓸 때, 우리는 직접 데이터가 제대로 입력됐는지 일일이 확인하지 않아도 됩니다!

1. Django가 제공하는 [redacted]을 통해 입력 데이터가 정확한지 확인합니다.
2. 입력된 정보를 자동으로 [redacted]클래스의 [redacted]메서드를 통해 데이터베이스에 저장하고, 사용자에게 결과를 응답합니다.
3. 사용자가 잘못된 정보를 입력하면 Django의 Form 클래스가 제공하는 [redacted] 메서드를 통해 자동으로 오류 메시지를 전달해줍니다.

- 이번 시간이 끝나면, Django의 Form 및 ModelForm 클래스를 통해 복잡한 유효성 검사 로직을 작성하지 않고도 웹페이지가 정확하게 데이터를 처리하도록 만들 수 있습니다.

학습 목표

- ✓ Django Form이 무엇인지 설명할 수 있다.
- ✓ Form을 통해 사용자 입력 데이터를 수집하고 처리할 수 있다.
- ✓ Form 클래스 정의 및 유효성 검사를 직접 수행할 수 있다.
- ✓ ModelForm을 활용하여 데이터베이스와 연동되는 입력 폼을 생성할 수 있다.
- ✓ Form과 ModelForm의 차이를 이해하고, 적절한 상황에 따라 선택하여 활용할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Django Form

HTML Form의 한계

HTML ‘form’

지금까지 사용자로부터 데이터를 제출받기 위해 활용한 방법
그러나 비정상적 혹은 악의적인 요청을 필터링 할 수 없음

- 유효한 데이터인지에 대한 확인이 필요합니다.

유효성 검사란?

유효성 검사

수집한 데이터가 정확하고 유효한지 확인하는 과정

- Django Form의 유효성 검사는 사용자가 입력한 데이터가 올바른 형식인지 자동으로 점검하는 기능을 제공합니다.
- 예를 들어, 필수 입력 값이 비어 있거나 잘못된 이메일 형식을 입력하면 오류를 알려줍니다.
- 이 과정을 통해 서버에 잘못된 데이터가 저장되지 않도록 보호할 수 있습니다.

유효성 검사 구현의 어려움

- 유효성 검사를 구현하기 위해서는 입력 값, 형식, 중복, 범위, 보안 등 많은 것들을 고려해야 함
- 이런 과정과 기능을 직접 개발하는 것이 아닌 Django가 제공하는 Form을 사용

Form Class

Django Form의 기능

Django Form

사용자 입력 데이터를 수집하고,
처리 및 유효성 검사를 수행하기 위한 도구

- ※ 유효성 검사를 단순화하고 자동화 할 수 있는 기능을 제공합니다.
- ※ 사용자가 잘못 입력한 데이터는 자동으로 오류로 처리되어 안전성을 높입니다.
- ※ 개발자는 이를 통해 빠르고 일관된 입력 검증 기능을 구현할 수 있습니다.

Form Class 정의

- Form Class를 상속받아 내용과 제목에 대한 사용자 입력을 받는 ArticleForm을 정의하는 방법

```python
# articles/forms.py

from django import forms


class ArticleForm(forms.Form):
    title = forms.CharField(max_length=10)
    content = forms.CharField()
```

Form Class를 적용한 new logic(1/3)

- view 함수 new 변경

```python
# articles/views.py

from .forms import ArticleForm


def new(request):
    form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

Form Class를 적용한 new logic(2/3)

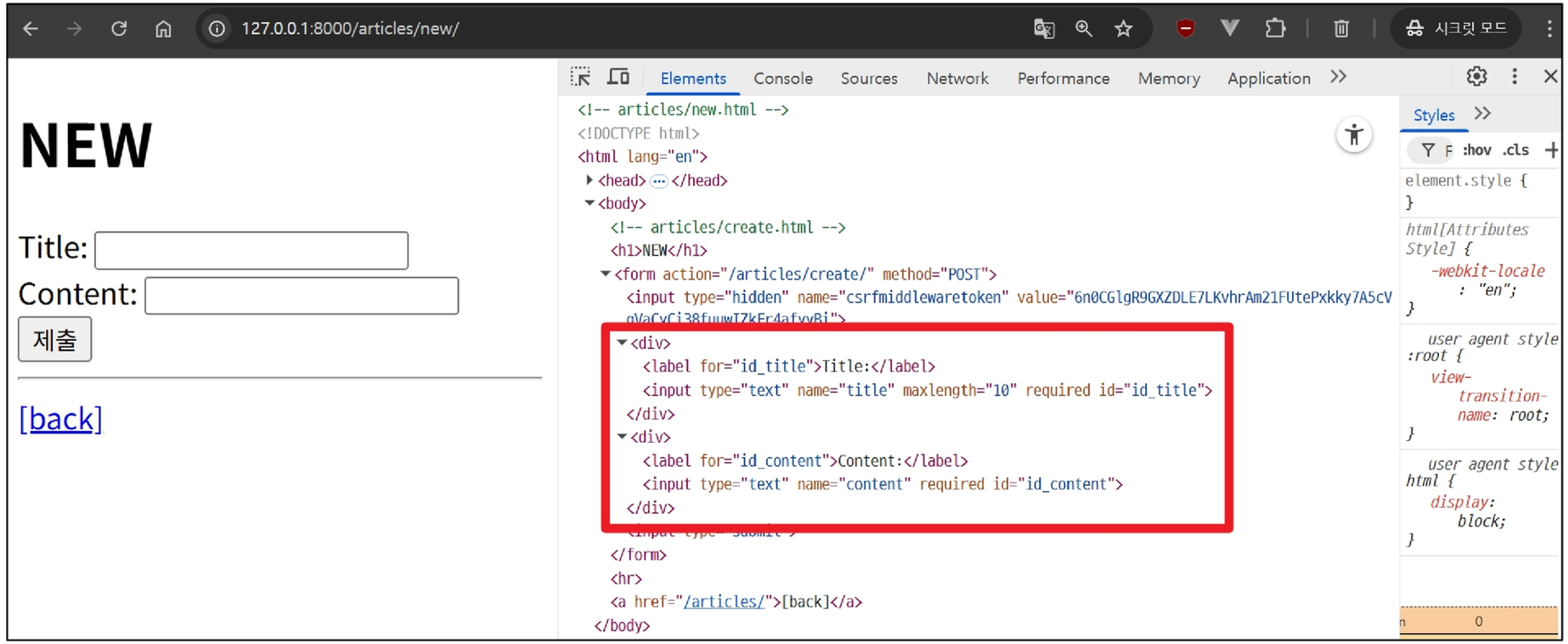
- new 페이지에서 form 인스턴스 출력

```
<!-- articles/new.html -->

<h1>NEW</h1>
<form action="{% url 'articles:create' %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

Form Class를 적용한 new logic(3/3)

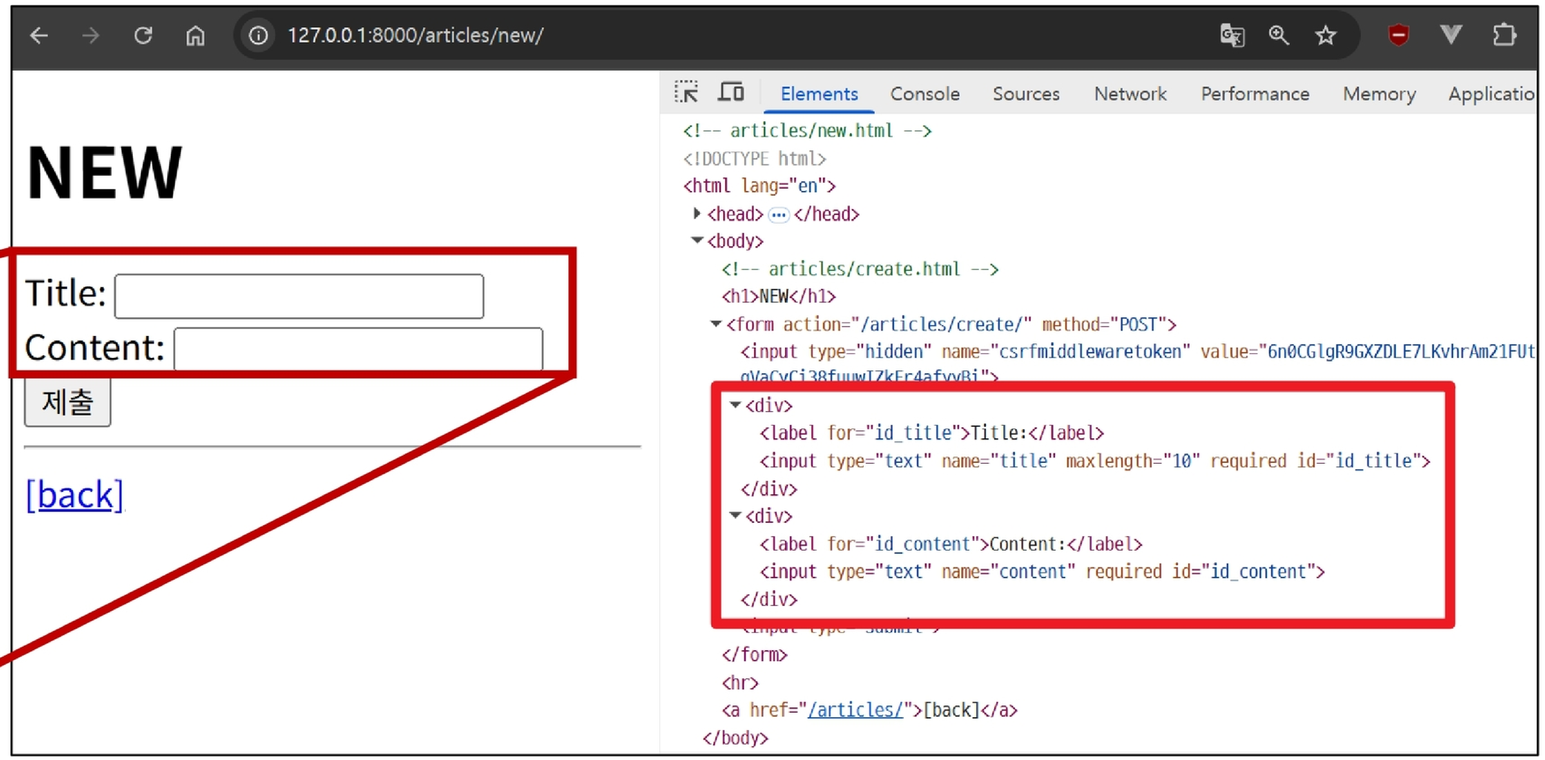
- new 페이지에서 form 인스턴스 출력



<그림1_Django_Form_Form class를 적용한 new logic>

Form class가 대체하는 것

```python
class ArticleForm(forms.Form):
    title = forms.CharField(max_length=10)
    content = forms.CharField()
```



<그림3_Django_Form_Form class 대체 예시>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Widgets

Widgets

Widgets
폼 필드를 화면에 표시하는 HTML 입력 요소를 정의하는 구성요소

<개념1_Django_Form_widgets 개념>

Widgets

HTML ‘input’ element의 표현을 담당

- Django Form의 widgets은 각 필드가 HTML에서 어떻게 렌더링 될지를 결정합니다.
- 예를 들어, `TextInput`, `Select`, `CheckboxInput` 등 다양한 위젯 클래스를 사용해 입력 방식과 속성을 세부 조정할 수 있습니다.

Widget 적용

- Widget은 단순히 input 요소의 속성 및 출력되는 부분을 변경하는 것

```python
# articles/forms.py

from django import forms


class ArticleForm(forms.Form):
    title = forms.CharField(max_length=10)
    content = forms.CharField(widget=forms.Textarea)
```

<그림4_Django_Form_widgets 적용 예시>

https://docs.djangoproject.com/ko/5.2/ref/forms/widgets/#built-in-widgets

Django ModelForm

Form vs. ModelForm

Form

사용자 입력 데이터를
DB에 저장하지 않을 때
(ex. 검색, 로그인)

ModelForm

사용자 입력 데이터를
DB에 저장해야 할 때
(ex. 게시글 작성, 회원가입)

ModelForm 기능

ModelForm

Model과 연결된 Form을 자동으로 생성해주는 기능을 제공

- ModelForm은 Form 클래스와 Model 클래스를 결합한 형태로, 모델 필드를 기반으로 입력 폼을 자동 생성해줍니다.
- 데이터 수집과 저장 과정을 동시에 처리할 수 있도록 도와줍니다.

ModelForm class 정의

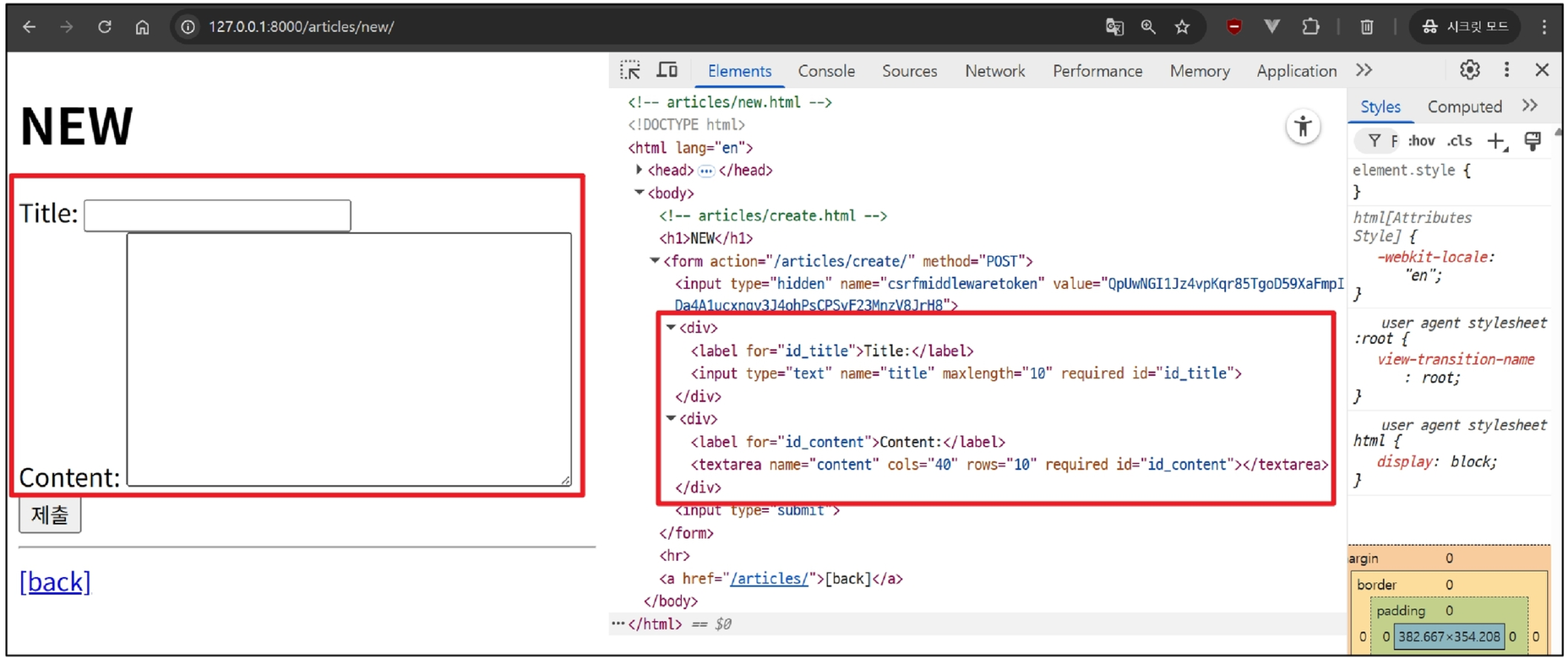
- 기존 ArticleForm 클래스 수정

```python
# articles/forms.py

from django import forms
from .models import Article


class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = '__all__'
```

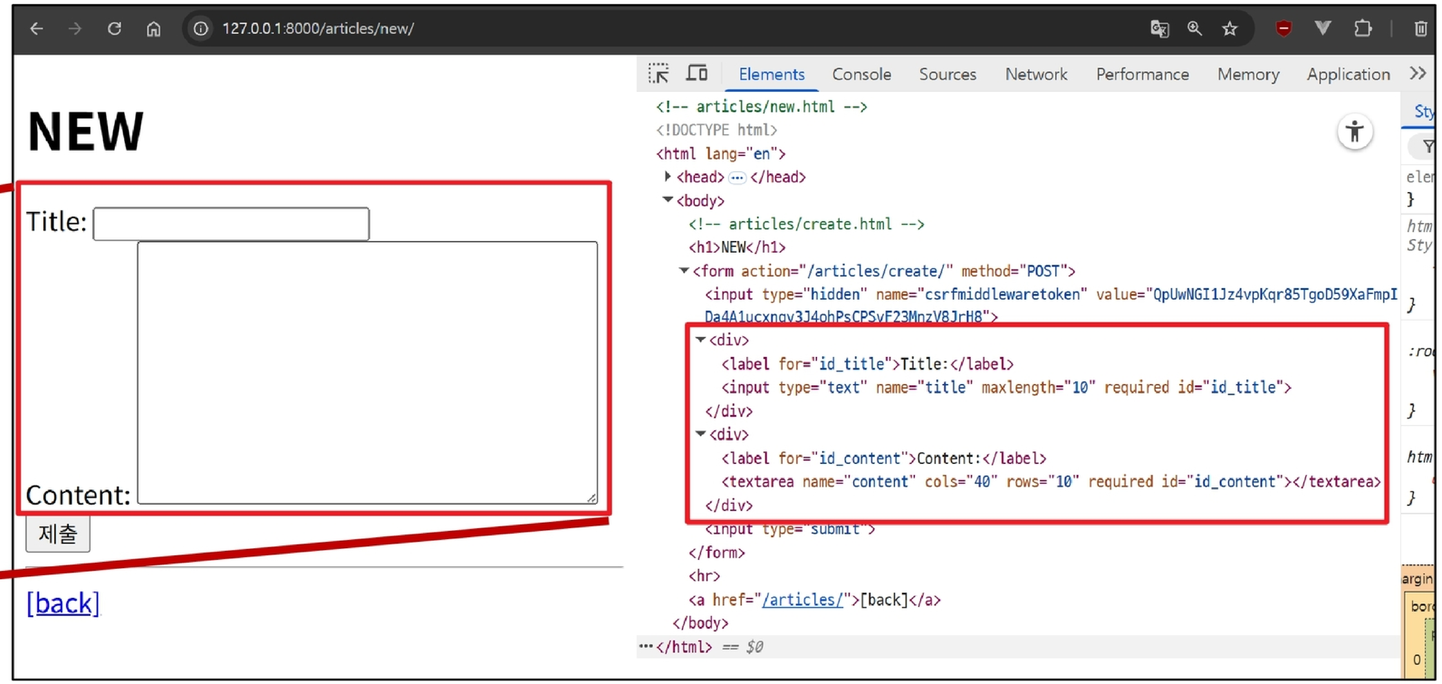
Django ModelForm



<그림5_Django_Form_ModelForm 적용 예시>

ModelForm class가 대체하는 것

```
class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = '__all__'
```



<그림6_Django_Form_ModelForm class 대체 예시>

Meta class

Meta class

Meta class

ModelForm의 정보를 작성하는 곳

- Meta class는 ModelForm 내부에서 어떤 모델과 연결할지,어떤 필드를 사용할지 등을 정의하는 설정 공간입니다.
- 폼의 동작 방식을 제어하는 핵심 역할을 합니다.

‘fields’ 및 ‘exclude’ 속성

- exclude 속성을 사용하여 모델에서 포함하지 않도록 필드를 지정할 수도 있음

```python
# articles/forms.py


class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = ('title',)
```

```python
# articles/forms.py


class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        exclude = ('title',)
```

Meta class 주의사항

- Django에서 `ModelForm`에 대한 추가 정보나 속성을 작성하는
  클래스 구조를 `Meta` 클래스로 작성 했을 뿐이며, 문법적으로는 클래스 안의 클래스(inner class)가 맞지만,
  상속하거나 객체를 만드는 용도가 아니라 `ModelForm`의 설정 값을 적어두는 공간으로 이해하기

ModelForm 적용

ModelForm을 적용한 create 로직 (1/2)

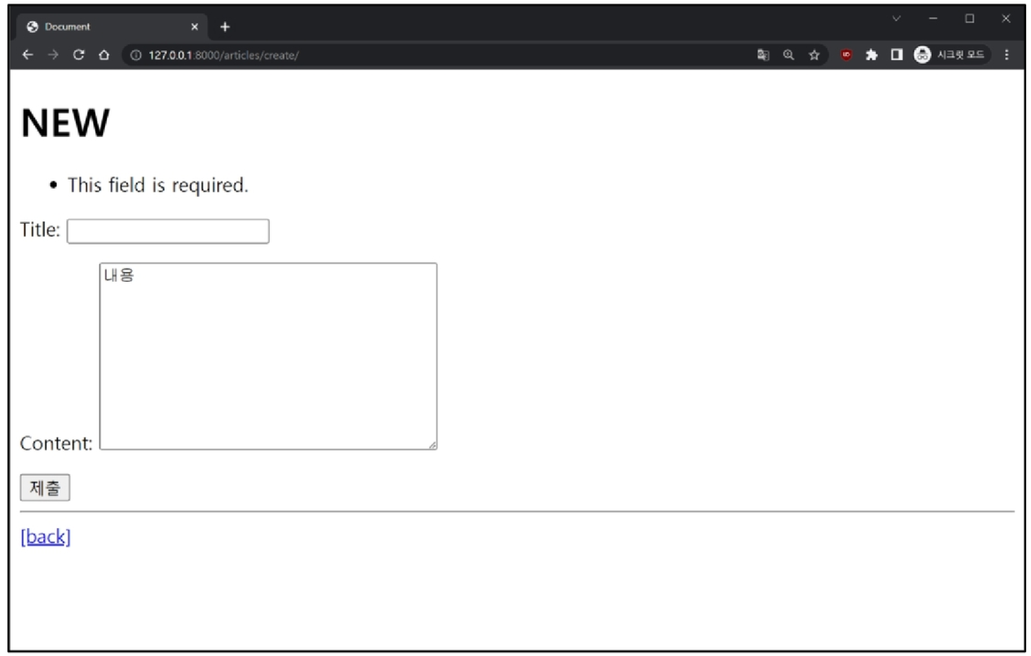
```python
# articles/views.py

from .forms import ArticleForm


def create(request):
    form = ArticleForm(request.POST)
    if form.is_valid():
        article = form.save()
        return redirect('articles:detail', article.pk)
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

ModelForm을 적용한 create 로직 (2/2)

- 제목 input에 스페이스바로 공백만 입력 후 제출 시 에러 메시지 출력 확인 ⇨ 유효성 검사의 결과



<그림7_Django_Form_ModelForm을 적용한 create 로직>

TIP

아무것도 입력하지 않고 제출하면 Django보다 브라우저가 먼저 "이 입력란을 작성하세요"라고 막습니다. Form이 input에 `required` 속성을 자동으로 붙여주기 때문입니다. 공백만 입력하면 브라우저 검사는 통과하지만, Django가 앞뒤 공백을 제거한 뒤 빈 값으로 판단해 에러를 돌려줍니다.

is_valid() 활용

is_valid()

여러 유효성 검사를 실행하고,
데이터가 유효한지 여부를 Boolean으로 반환

공백 데이터가 유효하지 않은 이유와 에러메시지가 출력되는 과정

- 별도로 명시하지 않았지만
  모델 필드에는 기본적으로 빈 값은
  허용하지 않는 제약조건이 설정 되어있음
- 빈 값은 is_valid()에 의해 False로
  평가되고 form 객체에는 그에 맞는
  에러 메시지가 포함되어 다음 코드로
  진행됨

```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

```python
# articles/views.py
def create(request):
    form = ArticleForm(request.POST)
    if form.is_valid():
        article = form.save()
        return redirect('articles:detail', article.pk)
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

ModelForm을 적용한 edit 로직

```python
# articles/views.py

def edit(request, pk):
    article = Article.objects.get(pk=pk)
    form = ArticleForm(instance=article)
    context = {
        'article': article,
        'form': form,
    }
    return render(request, 'articles/edit.html', context)
```

```html
<!-- articles/edit.html -->

<h1>EDIT</h1>
<form action="{% url 'articles:update' article.pk %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

ModelForm을 적용한 update 로직

```python
# articles/views.py

def update(request, pk):
    article = Article.objects.get(pk=pk)
    form = ArticleForm(request.POST, instance=article)
    if form.is_valid():
        form.save()
        return redirect('articles:detail', article.pk)
    context = {
        'article': article,
        'form': form,
    }
    return render(request, 'articles/edit.html', context)
```

save 메서드

개념

save()

데이터베이스 객체를
만들고 저장하는
ModelForm의 인스턴스 메서드

- form 데이터가 유효한 경우, save() 메서드를 호출하면 모델 인스턴스를 생성하고 데이터베이스에 저장됩니다.
- instance 인자를 통해 새 객체 생성과 기존 객체 수정도 구분할 수 있습니다.
- 이 과정을 통해 SQL을 작성하지 않고도 손쉽게 DB에 저장할 수 있습니다.

save() 메서드가 생성과 수정을 구분하는 법

- 키워드 인자 instance 여부를 통해 생성할 지, 수정할 지를 결정

```
# CREATE

form = ArticleForm(request.POST)
form.save()
```

```
# UPDATE

form = ArticleForm(request.POST, instance=article)
form.save()
```

Django Form 정리

- “사용자로부터 데이터를 수집하고 처리하기 위한 강력하고 유연한 도구”
- HTML `form`의 생성, 데이터 유효성 검사 및 처리를 쉽게 할 수 있도록 도움

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

HTTP 요청 다루기

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

View 함수 구조 변화

new & create view 함수간 공통점과 차이점

공통점

"데이터 생성을 구현하기 위함"

차이점

"new는 GET method 요청만을,
create는 POST method 요청만을 처리"

view 함수 구조화의 목적

HTTP request method 차이점을 활용해 동일한 목적을 가지는 2개의 view 함수를 하나로 구조화

new & create 함수 결합

new & create 함수 결합 (1/2)

```
def new(request):
    form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

```
def create(request):
    form = ArticleForm(request.POST)
    if form.is_valid():
        article = form.save()
        return redirect('articles:detail', article.pk)
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

new & create 함수 결합 (2/2)

```python
def new(request):
    form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

+

```python
def create(request):
    form = ArticleForm(request.POST)
    if form.is_valid():
        article = form.save()
        return redirect('articles:detail', article.pk)
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

→

```python
def create(request):
    if request.method == 'POST ' :
        form = ArticleForm(request.POST)
        if form.is_valid():
            article = form.save()
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

새로운 create view 함수 (1/5)

- new와 create view 함수의 공통점과 차이점을 기반으로 하나의 함수로 결합

```python
# articles/views.py

def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)
        if form.is_valid():
            article = form.save()
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

새로운 create view 함수 (2/5)

- 두 함수의 유일한 차이점이었던 request method 에 따른 분기

```
# articles/views.py

def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)
        if form.is_valid():
            article = form.save()
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

새로운 create view 함수 (3/5)

- POST일 때는 과거 create 함수 구조였던 객체 생성 및 저장 로직 처리

```python
# articles/views.py

def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)
        if form.is_valid():
            article = form.save()
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

새로운 create view 함수 (4/5)

- POST가 아닐 때는 과거 new함수에서 진행했던 form 인스턴스 생성

```
# articles/views.py

def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)
        if form.is_valid():
            article = form.save()
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

새로운 create view 함수 (5/5)

- context에 담기는 form은
  1. `is_valid()`를 통과하지 못해 에러메시지를 담은 `form`이거나
  2. `else` 문을 통한 `form` 인스턴스

```python
def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)
        if form.is_valid():  # 1
            article = form.save()
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm()  # 2
    context = {
        'form': form,
    }
    return render(request, 'articles/new.html', context)
```

기존 new 관련 코드 수정 (1/3)

- 사용하지 않게 된 new url 제거

```python
# articles/urls.py

app_name = 'articles'
urlpatterns = [
    path('', views.index, name='index'),
    path('<int:pk>/', views.detail, name='detail'),
    # path('new/', views.new, name='new'),
    path('create/', views.create, name='create'),
    path('<int:pk>/delete/', views.delete, name='delete'),
    path('<int:pk>/edit/', views.edit, name='edit'),
    path('<int:pk>/update/', views.update, name='update'),
]
```

기존 new 관련 코드 수정 (2/3)

- new 관련 키워드를 create로 변경

```
<!-- articles/index.html -->

<h1>Articles</h1>
<a href="{% url 'articles:create' %}">CREATE</a>
<hr>
{% for article in articles %}
  <p>글 번호: {{ article.pk }}</p>
  <a href="{% url 'articles:detail' article.pk %}">
    <p>글 제목: {{ article.title }}</p>
  </a>
  <p>글 내용: {{ article.content }}</p>
  <hr>
{% endfor %}
```

```
<!-- articles/create.html -->

<h1>CREATE</h1>
<form action="{% url 'articles:create' %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

기존 new 관련 코드 수정 (3/3)

- new 관련 키워드를 create로 변경

```
# articles/views.py

def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)
        if form.is_valid():
            article = form.save()
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm()
    context = {
        'form': form,
    }
    return render(request, 'articles/create.html', context)
```

request method에 따른 요청의 변화

(GET) `articles/create/` 게시글 생성 페이지를 줘!

(POST) `articles/create/` 게시글을 생성해줘!

edit & update 함수 결합

새로운 update view 함수

- 기존 edit과 update view 함수 결합

```python
# articles/views.py

def update(request, pk):
    article = Article.objects.get(pk=pk)
    if request.method == 'POST':
        form = ArticleForm(request.POST, instance=article)
        if form.is_valid():
            form.save()
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm(instance=article)
    context = {
        'article': article,
        'form': form,
    }
    return render(request, 'articles/update.html', context)
```

update

edit

기존 edit 관련 코드 수정 (1/2)

- 사용하지 않는 edit url 제거

```
# articles/urls.py

app_name = 'articles'
urlpatterns = [
    path('', views.index, name='index'),
    path('<int:pk>/', views.detail, name='detail'),
    # path('new/', views.new, name='new'),
    path('create/', views.create, name='create'),
    path('<int:pk>/delete/', views.delete, name='delete'),
    # path('<int:pk>/edit/', views.edit, name='edit'),
    path('<int:pk>/update/', views.update, name='update'),
]
```

기존 edit 관련 코드 수정 (2/2)

- edit 관련 키워드를 update로 변경

```
<!-- articles/detail.html -->

<h2>DETAIL</h2>
<h3>{{ article.pk }} 번째 글</h3>
<hr>
<p>제목: {{ article.title }}</p>
<p>내용: {{ article.content }}</p>
<p>작성 시각: {{ article.created_at }}</p>
<p>수정 시각: {{ article.updated_at }}</p>
<hr>
<a href="{% url 'articles:update' article.pk %}">UPDATE</a><br>
<form action="{% url 'articles:delete' article.pk %}" method="POST">
  {% csrf_token %}
  <input type="submit" value="DELETE">
</form>
```

```
<!-- articles/update.html -->

<h1>UPDATE</h1>
<form action="{% url 'articles:update' article.pk %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

참고

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

ModelForm의 키워드 인자 구성

ModelForm 키워드 인자 data와 instance 살펴보기

- `data`는 첫 번째 자리라서 `data=`를 쓰지 않고 값만 순서대로 넘길 수 있음
- instance는 9번째 자리라서 순서대로 넘기려면 앞의 8개를 모두 채워야 함

  => 그래서 **`instance=article`**처럼 이름을 붙여 넘김

```python
# ModelForm의 상위 클래스인 BaseModelForm의 생성자 함수 모습

class BaseModelForm(BaseForm):
    def __init__(self, data=None, files=None, auto_id='id_%s', prefix=None,
                 initial=None, error_class=ErrorList, label_suffix=None,
                 empty_permitted=False, instance=None, use_required_attribute=None,
                 renderer=None):
```

```python
# articles/views.py

form = ArticleForm(request.POST, instance=article)
```

https://github.com/django/django/blob/main/django/forms/models.py#L345

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Widgets 응용

Widgets 응용 (1/2)

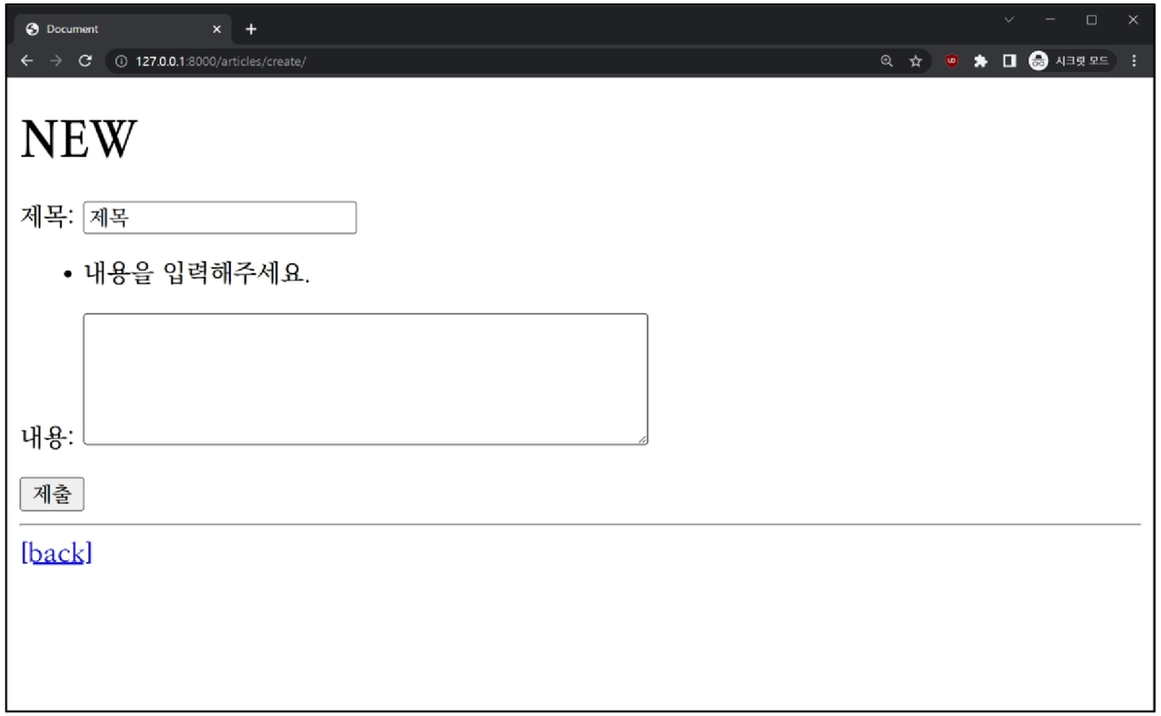
```python
# articles/forms.py

class ArticleForm(forms.ModelForm):
    title = forms.CharField(
        label='제목',
        widget=forms.TextInput(
            attrs={
                'class': 'my-title',
                'placeholder': 'Enter the title',
            }
        ),
    )

    class Meta:
        model = Article
        fields = '__all__'
```

Widgets 응용 (2/2)

```python
class ArticleForm(forms.ModelForm):
    title = forms.CharField(
        label='제목',
        widget=forms.TextInput(
            attrs={
                'class': 'my-title',
                'placeholder': 'Enter the title',
                'maxlength': 10,
            }
        ),
    )
    content = forms.CharField(
        label='내용',
        widget=forms.Textarea(
            attrs={
                'class': 'my-content',
                'placeholder': 'Enter the content',
                'rows': 5,
                'cols': 50,
            }
        ),
        error_messages={'required': '내용을 입력해주세요.'},
    )
```



<그림8_Django_Form_widgets 응용 예시>

※ 필드를 직접 선언하면 모델에서 자동으로 가져오던 설정(label, max_length 등)을 직접 다시 써야 합니다. `attrs`의 `maxlength`는 브라우저 입력 제한일 뿐이고, 서버 검증까지 원하면 `forms.CharField(max_length=10, ...)`처럼 써야 합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

필드를 수동으로 렌더링

필드를 수동으로 렌더링 하기

```
{{ form.non_field_errors }}
<form action="..." method="POST">
  {% csrf_token %}
  <div>
    {{ form.title.errors }}
    <label for="{{ form.title.id_for_label }}">Title:</label>
    {{ form.title }}
  </div>
  <div>
    {{ form.content.errors }}
    <label for="{{ form.content.id_for_label }}">Content:</label>
    {{ form.content }}
  </div>
  <input type="submit">
</form>
```

실습

Django Form

- 2688. Form widget 사용법 예제
- 3234. 메뉴 관리 시스템 만들기 - widget 활용 1
- 3235. 메뉴 관리 시스템 만들기 - widget 활용 2

Django ModelForm

- 2686. ModelForm 기본 사용 예제
- 3034. Form과 ModelForm 작성
- 3035. Form과 ModelForm을 활용한 페이지 렌더링

HTTP 요청 다루기

- 2687. HTTP method 사용 예제
- 2689. 특정 HTTP method 요청 예제
- 3036. HTTP requests 관리와 코드 개선

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 Django Form이 하는 일로 옳지 않은 것은?

a) 데이터 입력 화면 생성

b) 데이터 유효성 검사

c) 데이터 자동 저장 처리

d) 데이터베이스에 접속하여 직접 테이블 삭제

2 ModelForm과 일반 Form의 차이에 대해 옳은 것은?

a) ModelForm은 DB와 연결되지 않는다.

b) 일반 Form은 DB에 데이터를 저장할 때 사용한다.

c) ModelForm은 Model과 연결되어 있어 입력 데이터를 DB에 저장할 수 있다.

d) 일반 Form은 자동 유효성 검사를 지원하지 않는다.

3 다음 중 Django Form의 유효성 검사를 수행하는 메서드는?

a) save()

b) is_valid()

c) redirect()

d) render()

4 Form의 입력 필드를 화면에 렌더링할 때 사용하는 옵션 중, 각 필드를 <p> 태그로 감싸주는 옵션은?

a) `{{form.as_ul}}`

b) `{{form.as_table}}`

c) `{{form.as_p}}`

d) `{{form.as_render}}`

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 Django에서 사용자 입력 데이터를 저장하거나 수정할 때 사용하는 메서드는?

a) update()
b) create()
c) save()
d) delete()

6 Django Form 클래스의 기본 구조에서 HTML의 입력 요소를 조정할 때 사용하는 것은?

a) model
b) widget
c) validator
d) queryset

7 Django의 ModelForm에서 사용할 필드를 선택적으로 제외할 때 사용하는 Meta 클래스 속성은?

a) fields
b) exclude
c) remove
d) drop

8 다음 중 Django Form에서 입력값이 유효하지 않을 때 발생하는 동작으로 가장 알맞은 것은?

a) 서버가 500 에러를 반환한다.
b) Form 객체가 생성되지 않는다
c) is_valid()는 False를 반환하고 오류 메시지가 form 객체에 저장된다
d) 잘못된 값도 그대로 저장된다.

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 Form 클래스에서 입력 필드에 placeholder나 class 등 추가 속성을 설정할 수 있는 곳은?

a) fields
b) model
c) attrs
d) exclude

10 다음 중 Django view 함수가 HTTP의 POST 요청을 처리하는 경우로 올바른 것은?

a) 화면에 폼을 출력하여 사용자에게 보여준다.
b) 사용자가 입력한 데이터를 받아서 유효성 검사를 수행한다.
c) 데이터베이스에 있는 데이터를 화면에 표시한다.
d) URL 주소를 통해 직접 화면에 접근할 때 사용된다.

확인 문제

정답 및 해설

1. d) 데이터베이스에 접속하여 직접 테이블 삭제 2. c) ModelForm은 … 되어 있다. 3. b) is_valid()

4. c) `{{form.as_p}}` 5. c) save() 6. b) widget

1. Django Form은 사용자로부터 입력받은 데이터의 화면 표시, 유효성 검사, 데이터 처리를 담당하지만, 데이터베이스의 테이블을 직접 삭제하는 기능은 제공하지 않습니다. 테이블 삭제와 같은 작업은 데이터베이스 관리자의 권한으로 Django의 Form 클래스와는 관련이 없습니다.
2. ModelForm은 Django의 모델(Model)과 직접 연결되어 있어 데이터베이스에 저장하거나 수정할 때 편리하게 사용할 수 있습니다. 일반 Form은 Model과 연결되지 않고 단지 입력값 수집 및 유효성 검사만 처리합니다.
3. Django Form의 is_valid() 메서드는 사용자 입력 데이터가 유효한 형식인지 검사하는 기능을 제공합니다. save() 메서드는 유효성 검사를 통과한 후 데이터를 저장할 때 사용하는 메서드입니다.

4. Django의 Form 클래스에서 {{ form.as_p }}를 사용하면 각 입력 필드를 자동으로 <p> 태그로 감싸서 화면에 출력합니다. 이 외에 as_ul, as_table 같은 다른 옵션도 존재하지만, <p> 태그를 사용하는 것은 as_p입니다.

5. Django의 ModelForm은 save() 메서드를 사용하여 데이터를 데이터베이스에 저장하거나 기존 데이터를 수정합니다. create(), update(), delete()는 Django 모델의 메서드로 사용될 수 있지만 Form에서 데이터를 처리할 때는 사용하지 않습니다.

6. Form 클래스에서 HTML 입력 요소의 스타일이나 추가 속성(placeholder, class 등)을 지정할 때 Widget을 사용합니다. Widget을 통해 입력 필드가 HTML에서 어떻게 렌더링될지 세부적으로 조정할 수 있습니다.

확인 문제

정답 및 해설

7. b) exclude 8. c) is_valid()는 False를 반환하고 오류 메시지가 form 객체에 저장된다. 9. c) attrs 10. b) 사용자가 입력한 데이터를 받아서 유효성 검사를 수행한다.

7. Django ModelForm에서 Meta 클래스의 exclude 속성은 폼에서 제외하고 싶은 필드를 지정하는 데 사용됩니다. 반면 fields 속성은 포함할 필드를 명시적으로 정의할 때 사용합니다.

8. Django Form에서 is_valid()는 입력값이 유효하지 않으면 False를 반환합니다. 이때 발생한 오류 내용은 form 객체의 errors 속성에 저장되어 사용자에게 출력될 수 있습니다. 이 과정은 자동으로 수행되며 서버 에러가 발생하지 않고, 잘못된 값도 저장되지 않습니다.

9. 입력 필드에 추가 속성(placeholder, class 등)을 지정하려면 Form 클래스의 Widget 내부의 attrs 속성을 사용합니다. 이를 통해 화면에 표시되는 HTML 요소를 세부적으로 제어할 수 있습니다.

10. HTTP POST 요청은 사용자가 입력한 데이터를 서버로 전송하여 처리할 때 사용됩니다. Django에서 POST 요청은 사용자가 제출한 폼 데이터를 받아 유효성 검사를 거쳐 데이터베이스에 저장하거나 처리하는 데 활용됩니다.

활동 정리

핵심 키워드

개념	설명	예시
Form	사용자 입력을 처리하고 유효성 검사를 자동화하기 위한 Django 클래스	`class ArticleForm(forms.Form)`
is_valid()	입력 데이터의 유효성을 검사하여 True/False 반환	`if form.is_valid():`
save()	유효한 데이터를 데이터베이스에 저장	`form.save()`
ModelForm	모델과 연결된 폼을 자동 생성해주는 Django 클래스	`class ArticleForm(forms.ModelForm)`
Meta class	ModelForm 내부에서 model과 fields를 정의하는 내부 클래스	`class Meta: model = Article, fields = '__all__'`
Widget	폼 필드를 HTML 요소로 어떻게 보여줄지 정의하는 구성 요소	`forms.Textarea, attrs={'placeholder': '내용 입력'}` 등
instance	ModelForm에서 기존 데이터를 수정할 때 사용되는 인자	`form = ArticleForm(instance=article)`

<표1_Django_Form_핵심 키워드>

활동 정리

오늘 공부한 내용 요약 및 정리

- **Django Form**
  - 사용자 입력 데이터를 수집하고 유효성 검사를 자동으로 수행할 수 있는 클래스
  - HTML form과 달리 보안과 데이터 검증 기능을 제공함

- **Form 클래스 정의**
  - forms.Form 또는 forms.ModelForm을 상속받아 폼을 정의
  - 각 필드는 CharField, EmailField 등으로 구성함

- **유효성 검사: is_valid()**
  - 사용자 입력값의 형식을 검사
  - 오류가 있을 경우 자동으로 메시지를 포함한 form 객체 반환

- **ModelForm**
  - 모델과 연결된 폼 클래스로 DB 연동이 자동화됨
  - Meta 클래스에서 model, fields를 명시함

활동 정리

“웹사이트에서 회원가입이나 글을 쓸 때, 우리는 직접 데이터를 검증하지 않아도 됩니다!”

1. Django가 제공하는 **Form**을 통해 입력 데이터가 정확한지 확인합니다.
2. 입력된 정보를 자동으로 **ModelForm** 클래스의 **save()** 메서드를 통해 데이터베이스에 저장하고, 사용자에게 결과를 응답합니다.
3. 사용자가 잘못된 정보를 입력하면 Django의 Form 클래스가 제공하는 **is_valid()** 메서드를 통해 자동으로 오류 메시지를 전달해줍니다.

- Django의 Form 시스템을 활용하면, 복잡한 로직 없이도 안전하고 신뢰할 수 있는 사용자 입력 처리가 가능합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다